

Microsoft Discovery Core Capabilities Hands-on Guide

English · As of 2026-09-25 · Azure cloud service

One goal: build a research workflow that finds evidence, verifies it with real computation, and evaluates the results again when a human changes the constraints.

Validation boundary: This guide cross-checks official documentation and locally verifies the supplied data and calculations. It does not claim that the end-to-end Discovery labs have run in this Azure account. This revision is documentation-only; it does not deploy Azure resources or record a new execution.

S00 — Start here

Microsoft Discovery cloud became **generally available on 2026-06-02**. The separate **Discovery desktop app and unified Workbench are in Preview** and are not required for this guide. Use the **Discovery Engine**, not deprecated static `kind: workflow` agents. Role names retain `(Preview)` where it is part of their official display name. [\[S01\]](#) [\[S03\]](#) [\[S10\]](#) [\[S28\]](#)

Audience	Route
Administrator preparing an environment	L00 → L01 → L02 → L03, then L05 steps 1–5 to prepare the tool
Researcher using a prepared environment	Confirm H01–H06 below → L04 → L05 steps 6–8 → L06 → L07 → L09
Team verifying collaboration and access	Add L08. Actual verification requires another test user
Developer extending tools	Add E01 and E02

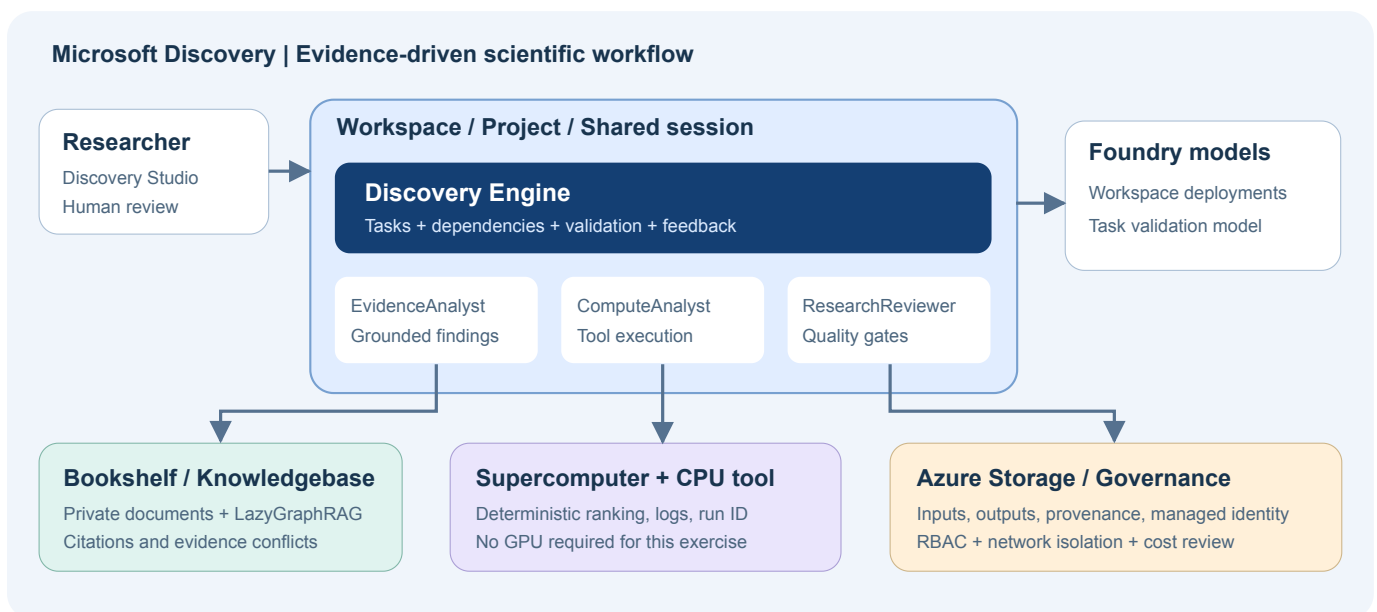
Role boundary: Bookshelf, node-pool, Registry, tool creation, and indexing preparation belong to administrators or explicitly authorized resource operators. Project Contributor alone does not authorize all these actions. Coordinate with an administrator rather than granting every researcher subscription Owner for convenience. [\[S20\]](#)

Researcher entry check: Start at L04 without repeating infrastructure creation when the administrator supplies these actual values and readiness evidence. Example names or a verbal assurance are not sufficient.

ID	Handoff and readiness evidence
H01	Actual Workspace/Project URLs and resource IDs, successful <code>thermalho1</code> , correct tenant
H02	Existing, usable <code>gpt-5-4</code> deployment
H03	<code>thermal-evidence</code> version <code>1</code> , four TXT sources, indexing <code>Succeeded</code>
H04	Actual <code>thermaldata</code> , <code>evidencepack / candidatecsv</code> Asset IDs, eight source CSV rows
H05	<code>thermal-ranking</code> Tool ID, definition version/image digest, full <code>cpulab</code> Node pool ID
H06	Researcher's project/model/tool/KB permissions and Blob/network access to inputs and outputs

Allow separate time for access approval, deployment, and indexing. Plan **approximately 2–3 hours** for the researcher exercises, not a guaranteed completion time. Every lab follows **Goal → Steps → Pass criteria**. Advance only after the relevant prerequisites pass.

Keep the concepts separate. A Workspace is the shared infrastructure boundary, a Project is the research and access boundary, and a Shared session groups related research tasks. Documentation and APIs also use *investigation* for the associated research container. A Discovery Storage Container is a **storage-reference resource**, not the Azure Blob container itself. [\[S02\]](#) [\[S12\]](#) [\[S16\]](#)



S01 — Core capability coverage

The core route covers the four central capabilities—**agents, graph-based knowledge, computation, and the Discovery Engine**—plus their data, security, and collaboration requirements. E01/E02 extend the same platform and require separate checks of permissions and connected-service availability.

ID	Capability	Lab	Evidence to inspect
C01	Studio, Workspace, Project, Shared session	L01, L04	Resource IDs, successful state, session
C02	Storage Container/Asset; reading, writing, sharing files	L02, L04, L06	Source path, output Asset ID, file content
C03	Bookshelf, GraphRAG, indexing, citations and evidence comparison	L03, L04	Successful index and actual source citations
C04	Prompt Agents, models, structured responses, versions and reuse	L04	Deployment name, JSON fields, agent version
C05	Supercomputer, node pools, Action Tools, execution and logs	L05	Operation ID, success state, actual JSON
C06	Cognition, task dependencies, validation and retries	L06	Task hierarchy, execution history, validation comments
C07	Hypothesis/constraint changes, human feedback and resume	L07	Before/after results and a new operation ID
C08	Collaboration, project RBAC, linked-resource access	L08	Allowed and denied operations by another user
C09	Traceability, retained artifacts and reproducibility	L06, L09	Evidence/input/version/computation/output chain
C10	Managed identity, networking, encryption, cost and shutdown	L00, L01, L09	Access settings, remaining jobs/resources/cost
C11	Code-environment and Hybrid Tools	E01	Generated code, actual run logs and output
C12	External MCP/OpenAPI and scientific-model integration patterns	E02	MCP call evidence and integration-pattern selection

Out of scope: installing the desktop app, enabling Preview Workbench, evaluating every partner model, physical experiments or robotics, and GPU benchmarking. Do not present these as outcomes of this lab.

S02 — Scenario and expected results

Question: Recommend the top two of eight fictional heat-sink materials that meet every hard constraint. Resolve conflicting documents using the latest correction, and never invent missing evidence.

All data was authored specifically for this exercise and is **synthetic**. It is not real supplier information, published research, measurements, or safety certification. Both editions use the same English source corpus; the guide and requested agent-response language differ.

File	Purpose
data/bookshelf/01_requirements.txt	REQ-001: requirements and scoring formula
data/bookshelf/02_candidate_catalog.txt	CAT-001: initial candidate values
data/bookshelf/03_screening_results.txt	TEST-001: corrosion status and missing evidence
data/bookshelf/04_revision_notes.txt	REV-001: GAMMA correction and follow-up constraint
data/materials.csv	Resolved computational input; not independent additional evidence
scripts/score_materials.py	Reference calculator using only the standard library
scripts/verify_ranking.py	Validates the complete downloaded raw ranking-file contract, not its execution location
tools/thermal-ranking/	CPU-tool Dockerfile and definition template
config/lab.json	Account, subscription, and region for this lab

Property	Hard requirement
Thermal conductivity	$\geq 180 \text{ W/(m}\cdot\text{K)}$
Density	$\leq 3.00 \text{ g/cm}^3$
Material cost	$\leq 5.00 \text{ USD/kg}$
Recycled content	$\geq 50\%$
Corrosion screening	pass ; unknown is not a pass

Score only eligible candidates. `recycled_pct` is a value from 0 to 100. Round the final score to one decimal place and break ties by candidate ID in ascending order.

```
score = 50 * min(conductivity_w_mk / 250, 1)
        + 30 * max(0, 1 - cost_usd_kg / 5)
        + 20 * min(recycled_pct / 100, 1)
```

Candidate	Baseline decision	Score / reason
ALPHA	eligible	67.8
BETA	excluded	Density and cost are too high; recycled content is too low
GAMMA	excluded	Use 175 from REV-001; conductivity is too low
DELTA	eligible	68.6

Candidate	Baseline decision	Score / reason
EPSILON	excluded	Cost exceeds the limit
ZETA	needs_review	No corrosion evidence
ETA	excluded	Recycled content is too low
THETA	eligible	60.8

Baseline: DELTA → ALPHA → THETA; recommend DELTA/ALPHA. **Follow-up:** lower only the cost ceiling to 3.00; DELTA is the sole eligible candidate, still scoring 68.6. Keep the cost-normalization denominator at 5.

Decision order: Any known hard-constraint failure means `excluded`; otherwise missing evidence means `needs_review`; the remaining candidates are `eligible`. Under the follow-up ceiling, ZETA is therefore `excluded` on cost while its corrosion evidence remains missing.

L00 — Prepare access, cost and roles

Goal: Treat service approval, Azure permissions, networking, and spending approval as separate prerequisites.

Steps:

1. Match the signed-in account to `config/lab.json`. This lab targets `junwoojeong@MngEnvMCAP757124.onmicrosoft.com`, subscription `51531604-2337-4c05-bc05-3c3d4ff154e5`, and tenant `46e9cdaa-fed3-4131-aa28-c1fc8a8a043a`. Do not rely on a different default subscription.
2. Confirm with your Microsoft representative that the subscription is **enabled for Discovery**. Spending approval and Contributor permissions do not replace the service allow-list. [\[S03\]](#) [\[S04\]](#)
3. In Azure Portal → Subscriptions → your subscription → Resource providers, check/register `Microsoft.Discovery` and the dependencies listed in the official guide. Registration requires appropriate subscription permissions. The following command is read-only.

```
az provider show --namespace Microsoft.Discovery \
  --subscription 51531604-2337-4c05-bc05-3c3d4ff154e5 \
  --query "{state:registrationState,types:resourceTypes[].resourceType}"
```

4. Check availability of actual resource types such as Workspaces, not just `Registered`. A `Pending` feature registration needs service-side clarification. Do not enable arbitrary development features. This account's earlier `DefaultFeature` request is recorded separately; it is not a universal installation procedure for every subscription.
5. Have an administrator prepare the Platform Administrator persona and the managed identity's least-privilege roles. **Permission to assign roles is separate.** Verify the actual Owner/RBAC Administrator/User Access Administrator or equivalent authority. Researcher and reviewer project roles are covered in L08.

6. The administrator reviews/configures the subscription-level **Discovery NSP Perimeter Joiner + Reader** assignments for the Discovery control-plane service App (92c174ac-8e41-4815-a1b7-d81b19ab03ce) using the official procedure. Do not assume ordinary Contributor includes role-assignment authority. ^[S18]
7. Choose a supported production region: **East US / Sweden Central / UK South**. This guide uses `swedencentral` . Check model/VM quota and always-on costs in A01.
8. Complete normal MFA/security-key sign-in. For private access, prepare VPN/ExpressRoute and Blob connectivity. Verify Portal sign-in, Studio access, and opening Blob outputs separately.

Pass criteria: Service access, providers, roles, quota, client connectivity, and spending approval are ready. If any is blocked, do not deploy; have the responsible owner resolve it.

L01 — Create the Workspace and Project

Goal: Prepare shared infrastructure for the researcher labs. For an existing environment, verify the pass criteria instead of redeploying. ^[S03] ^[S18]

Steps:

1. Use the dedicated RG `rg-discovery-hol-20260925` . Check global uniqueness of Workspace, storage-account, and Bookshelf names in the portal. The Workspace uses lowercase letters; Project `thermalhol` is lowercase and at most 12 characters. Names below are **proposals**, not already-created resources.
2. Create VNet `vnet-discovery-hol` . `10.80.0.0/16` is illustrative; replace it with an approved range if it overlaps existing networks.

Subnet	Example CIDR	Configuration
<code>supercomputerNodepoolSubnet</code>	<code>10.80.1.0/24</code>	Microsoft.Storage service endpoint
<code>aksSubnet</code>	<code>10.80.2.0/24</code>	Microsoft.Storage service endpoint
<code>workspaceSubnet</code>	<code>10.80.3.0/24</code>	Microsoft.App/environments delegation + Storage endpoint
<code>privateEndpointSubnet</code>	<code>10.80.4.0/24</code>	Dedicated to the Workspace
<code>agentSubnet</code>	<code>10.80.5.0/24</code>	Microsoft.App/environments delegation + Storage endpoint
<code>searchSubnet</code>	<code>10.80.6.0/24</code>	Dedicated to Bookshelf; Microsoft.App/environments delegation
<code>bookshelfPeSubnet</code>	<code>10.80.7.0/24</code>	Dedicated to Bookshelf

3. Create UAMI `id-discovery-hol` and grant Discovery Platform Contributor and Storage Blob Data Contributor at the necessary scopes. Configure image-pull access when the Registry is ready in L05. Do not share the subnets with other Workspaces/Bookshelves.
4. Create a Standard LRS storage account. Disable anonymous Blob access and allow only the required VNet/client paths. Create Blob containers `discoveryinputs` and `discoveryoutputs` . Follow the official

CORS configuration: `https://studio.discovery.microsoft.com` , `https://vscode.dev` , `https://*.vscode-cdn.net` ; GET/HEAD/PUT/DELETE/OPTIONS; headers `*` ; Max age 200. CORS does not replace authentication.

5. In **Microsoft Discovery Supercomputers → Create**, configure the VNet, `aksSubnet` , system SKU `Standard_D4s_v6` , and Cluster/Kubelet/Workload identity. Then use **Settings → Node pool → Create** to create `cpulab` . Where supported, target min 0 / max 1 and GPU 0 for the user tool pool. The system pool is separate.
6. In **Microsoft Discovery Workspaces → Create**, connect the subnets, UAMI, and Supercomputer. Keep default network isolation and verify your access path. Do not automatically add Preview Workbench tags or `NetworkIsolation=false` . Select MMK/CMK according to policy; CMK needs additional Key Vault and access configuration.
7. In Workspace **Settings → Chat Model Deployments**, verify that the validation deployment `gpt-5-4` / **model gpt-5.4** actually exists. Create it if missing. Distinguish it from the automatically provisioned cognition deployment. Do not replace it arbitrarily when adding other agent models.
8. In Studio `https://studio.discovery.microsoft.com` → **Data → Storage Containers (new) → Create Container**, create `thermaldata` and link the storage account. This registers a reference; it does not copy data.
9. In **Workspaces → your Workspace → Create Project**, create `thermalho1` and link the Storage Container. An administrator grants Foundry User on the Workspace managed resource group (MRG) to users who need the related advanced configuration.
10. Record resource IDs and MRGs. Wait for **Succeeded** , not merely **Accepted** . Finalize Discovery-resource tags before creation because the documentation states they cannot be changed afterward.

Pass criteria: Workspace, Supercomputer, CPU pool, Storage Container, and Project are ready; the required model exists. You can open the project in Studio, and both the user and managed identity have their respective required Blob access.

L02 — Register data and move files through the workflow

Goal: Register Blob files and Discovery references, preparing the inputs that agents will use later. [\[S15\]](#) [\[S16\]](#) [\[S17\]](#)

Steps:

1. Use Azure Portal's Storage account → Containers or an approved storage tool to upload the four TXT files to `discoveryinputs/bookshelf/` and the CSV to `discoveryinputs/compute/` . **Current documentation does not support directly uploading local files through Studio.**
2. Under Discovery Storage Container `thermaldata` , register Storage Assets `evidencepack` and `candidatecsv` . Use Data paths `discoveryinputs/bookshelf/` and `discoveryinputs/compute/materials.csv` , respectively. Verify the RG, region, and backing account.
3. Use the creation form's Storage Asset tab or the [official Storage Asset registration procedure](#) . The resource-group-level Storage Container/Assets are **referenced** by the project. Check the backing account and both Assets' paths/states.
4. Download the actual source CSV to verify eight candidates and GAMMA=175; record the Asset IDs for later attachment. Copy returned `discovery://` URIs and Asset IDs; do not invent them.

5. Agent reading/writing/sharing occurs in L04, and Task-input attachment in L06. Typing a path in a prompt alone does not establish a data connection.

Data path	Support and constraints
Bookshelf indexing	Use only the four supported TXT files in this lab. Do not put CSV/Python computation inputs into the knowledge index
Built-in file tools	Text such as TXT/MD/CSV/JSON/PY/YAML. <code>PreviewResource</code> returns only the first 30 KB
Binary files such as PDF/images	Support for some Bookshelf document ingestion is separate from native <code>PreviewResource</code> , which cannot read these binaries. Use a suitable custom tool
User opening an output	Managed-identity write access does not give the user Blob data permissions, firewall access, or CORS

Pass criteria: Source Blob files/content, the Storage Container's backing account, and both input Assets' actual IDs, paths, and successful state are verified. This lab can finish without an agent or computation tool.

L03 — Index a Bookshelf knowledgebase

Goal: Build the knowledge index for cross-document reasoning. Actual retrieval and citation verification follow in L04, after the index is ready. [\[S06\]](#) [\[S07\]](#)

Steps:

1. In **Microsoft Discovery Bookshelves** → **Create**, choose the same subscription/RG/region, dedicated `searchSubnet / bookshelfPeSubnet`, and UAMI. Set `indexSize=small` at creation. The current documented limit is **one Knowledgebase per Bookshelf**.
2. In Studio → **Resources** → **Knowledge** → **select Bookshelf** → **Create new**, create `thermal-evidence`, version `1`. Describe it in Description/Copilot instruction as evidence for synthesizing fictional heat-sink requirements, candidate claims, gaps, and authoritative corrections.
3. Connect `thermaldata`, `evidencepack`, and the same UAMI.
4. Prepare the memory-intensive **indexing pool** `indexlab` using the requirements confirmed in A01. Do not reuse the small D4 computation pool as if it met indexing requirements.
5. In Knowledgebase Details, verify Provisioning State= `Succeeded` and Status= `NotStarted`. Select **Index** → **Project** `thermalhol` → `indexlab` → **Start Indexing**.
6. Wait for the actual indexing Status= `Succeeded`. This is separate from the Azure resource's Provisioning State.
7. Record KB name/version, the linked source Asset, indexing operation, and successful state. Attach the querying agent in the next lab.

Pass criteria: The actual source link points to all four TXT files and KB indexing reports `Succeeded`. Indexing success alone does not verify retrieval quality or citations.

Updating the index: Only the initial full index is required. Official indexing/update guidance conflicts; follow A01. Do not assume deleting a Blob removes the corresponding KB document.

L04 — Specialist agents, models and versions

Goal: Create Prompt Agents with explicit roles, models, tools, and KBs, then interact with them directly. [\[S08\]](#)
[\[S09\]](#) [\[S10\]](#) [\[S25\]](#)

Steps:

1. In the project → **Resources** → **+ beside Agents** → **Create new agent**, create the three agents below.
 Use type Agent/Prompt Agent and Chat model deployment `gpt-5-4`.
2. Add the role instructions below. The default Discovery agent is for product guidance, not a substitute for these research agents.
3. Under **Knowledge Bases**, attach `thermal-evidence` to EvidenceAnalyst and ResearchReviewer.
 Prepare only ComputeAnalyst's model and instructions here; its first tool attachment/run occurs in L05.
4. Create a **New shared session** and attach L02's actual input Assets using the data controls. Invoke EvidenceAnalyst/ResearchReviewer through `@AgentName` or the agent selector.
5. Have an agent discover the CSV through `GetResourceContext`, read it through `PreviewResource`, and use `WriteResource` to create `data-check.md` confirming eight candidates and GAMMA=175. Use `ShareResource` when needed, then open the actual output Asset.

EvidenceAnalyst instructions

You analyze evidence for fictional heat-sink materials.
 Use connected thermal-evidence and attach actual source citations to every quantitative claim and decision. Follow REQ-001; prefer GAMMA=175 from REV-001.
 Never turn ZETA's unknown corrosion status into a pass or invent measurements.
 Separate synthetic inputs, computations, interpretation, and unexecuted work.
 Reply in English.

ComputeAnalyst instructions

You execute the thermal-ranking tool.
 Use rank_baseline for the original case and rank_cost_revision for the 3 USD/kg case.
 Report the actual operation ID, terminal state, logs, and output JSON Asset.
 Never substitute mental arithmetic or local reference results for cloud execution.
 If the tool is missing or fails, state the blocker. Reply in English.

ResearchReviewer instructions

You independently compare documentary evidence with actual computation JSON when available.
 Without computation results, review evidence only and mark arithmetic verification incomplete.
 Check all eight candidates, constraints, the GAMMA correction, ZETA's missing evidence, rankings, and scores. Produce a report with method, recommendation, exclusion reasons, evidence gaps, and limitations.
 Discover files with GetResourceContext and read them with PreviewResource.
 Never claim real experiments or safety certification. Reply in English.

6. Send the following to EvidenceAnalyst. Ask about **relationships and implications across the corpus**, rather than isolated values, to exercise Bookshelf's strengths.

@EvidenceAnalyst

Synthesize all documents in thermal-evidence to explain the tradeoffs among performance, cost, recycled content, and missing evidence that affect selection. Include a decision table for all eight candidates and information that could change the conclusions. Resolve GAMMA using REV-001 and retain ZETA as unknown. Link every quantitative claim and decision to an actual retrievable source citation. Defer final arithmetic ranking verification until the separate computation tool returns.

7. Ask ResearchReviewer to return the evidence checks as JSON. This is the **expected shape of the agent's evidence summary**, not the computation tool's `ranking.json`. Compare the actual response with the sources; do not copy the example and present it as execution evidence.

```
{
  "candidate_count": 8,
  "gamma_conductivity_w_mk": 175,
  "missing_evidence": [
    {
      "candidate_id": "ZETA",
      "source_field": "corrosion_status",
      "observed_value": "unknown"
    }
  ],
  "synthetic_data": true,
  "arithmetic_verified": false
}
```

Requesting JSON is not enforcing JSON Schema. Configure schema-constrained output explicitly if the model/UI supports it, and record which method was used. No computation has run at this stage, so do not set `arithmetic_verified` to `true`.

8. Improve and save the instructions once, then record the new version. Versions are immutable; distinguish history from the version actually used. Use **Copy from project** when reusing an agent in another accessible project. This creates an independent copy, not a live synchronized reference.

Pass criteria: Verify all three agents' models/instructions, both evidence agents' KB bindings, actual file reading/writing, at least three source citations, a structured evidence response, and a new version. ComputeAnalyst's tool run is the next lab's criterion. Do not set unsupported `temperature / top_p` for reasoning models. Chat and Task contexts are separate; repeat important inputs and constraints in the Task.

L05 — Run the actual CPU tool

Goal: Verify results using a real Supercomputer execution, logs, and files—not an LLM's statement that it calculated them. [\[S21\]](#) [\[S22\]](#) [\[S23\]](#) [\[S32\]](#)

Steps:

Steps 1–5 are administrator preparation; steps 6–8 are the researcher run.

1. These commands produce **local reference results**, not evidence of an Azure run.

```
python3 scripts/score_materials.py --input data/materials.csv \
  --output artifacts/reference-ranking.json
python3 scripts/score_materials.py --input data/materials.csv --max-cost 3 \
  --output artifacts/reference-ranking-cost3.json
```

2. Use an approved existing ACR or have an administrator create one for the lab. Builder/push permissions differ from the Supercomputer identity's pull permissions. Use roles appropriate to the Registry's RBAC/ABAC mode.
3. From the repository root, replace `<acr-name>` and run the following. **az acr build uploads the context and performs a billable cloud build in Azure ACR Tasks; it is not a local build.** It does not require a local Docker daemon.

```
az acr build --registry '<acr-name>' \
  --subscription 51531604-2337-4c05-bc05-3c3d4ff154e5 \
  --file tools/thermal-ranking/Dockerfile \
  --image thermal-ranking:1.0.0 .
```

4. Verify that `.dockerignore` allows only the calculator, synthetic CSV, and Dockerfile. Record successful build status and the Registry image/digest. A version tag can still be overwritten; its name alone does not guarantee immutability.
5. Copy `tools/thermal-ranking/tool-definition.template.json` and replace only `<acr-login-server>` in the copy. In Portal → **Microsoft Discovery Tools** → **Create** → **Basics**, set Name= `thermal-ranking`, Region= `swedencentral`, **Definition content file**=the edited JSON, and **Definition content version**= `1.0.0`. Leave Environment variables empty for this tool. Retain `worker`, both actions, and `/outputs`; verify `Succeeded` and the Tool ID after creation. The definition's `version` and the portal's Definition content version are separate fields. For REST, the definition object belongs inside `properties.definitionContent`. ^[S33]
6. Attach the registered tool under ComputeAnalyst's **Tools**. For the first manual conversation, **Confirm before running tool** can demonstrate the approval pause. Before Engine exercises, explicitly choose the approval policy for the two reviewed synthetic actions so unattended approval does not stall execution. Do not silently disable an approval requirement. Then request one actual `rank_baseline` run on `cpulab`. **This tool uses the synthetic CSV baked into its image.** It does not automatically mount the L02 Blob CSV; input changes require a rebuilt image or an explicit tool change.
7. Inspect the operation ID, `Succeeded`, actual `nodepoolId`, logs, and `ranking.json` Asset. `nodepoolId` must match the full `cpulab` ID from H05; putting a pool name in a prompt is not verification. `auto_promote: true` configures output sharing. Do not count a Foundry Code Interpreter run as a Supercomputer run. ^[S32]
8. Download the **original tool-produced file** and run the following check. Replace `<downloaded-ranking.json>` with its actual path. Do not substitute an agent's summary JSON or the local reference file.

```
python3 scripts/verify_ranking.py \
  --input '<downloaded-ranking.json>' --case baseline
```

The checker compares all eight candidates, decisions/reasons/gaps/sources, ranking/scores, constraints, and JSON structure. Missing/extra fields, incorrect order, duplicate keys, and invalid numbers fail. Success reports `content_verified: true` and `execution_verified: false`. File content cannot establish where execution occurred; separately verify the Azure operation, pool, logs, and Asset.

Pass criteria: Require the actual cloud operation ID, `cpulab`, logs/Asset, and a passing raw-JSON content check. Values must be `candidate_count=8`, `eligible_count=3`, and DELTA 68.6 → ALPHA 67.8 → THETA 60.8. `Accepted`, `NotStarted`, failure, and cancellation are not success. The first tool call may incur additional cold-start time.

L06 — Discovery Engine and the validation loop

Goal: Execute a research workflow with dependencies, independent validation, and file inheritance. [\[S11\]](#) [\[S12\]](#) [\[S13\]](#) [\[S14\]](#) [\[S15\]](#)

Steps:

1. Create a new Shared session in the same project. **Do not start the Engine yet.**
2. Create all Tasks below. Put the synthetic-data restriction, input Assets/KB/tool, output filenames, and validation requirements in each Description. Do not assume the Task knows earlier chat content.

Task	Parent	Depends on	Output and required validation
R0	—	—	<code>baseline-summary.md</code> : synthesize child results, recommend DELTA/ALPHA, include limitations
T1	R0	—	<code>evidence-map.md</code> : eight candidates, actual citations, GAMMA=175, ZETA=unknown
T2	R0	T1	Actual <code>rank_baseline</code> operation ID, logs, and JSON; eight candidates, three eligible, exact scores
T3	R0	T1, T2	<code>research-report.md</code> : compare sources with actual JSON; eight decisions, recommendation, gaps, method, limitations

3. **Parent expresses decomposition; Depends on expresses prerequisites.** Do not make R0 a dependency of T1 and create a cycle. Select an agent if the UI offers that control; otherwise add a Task comment naming the appropriate agent and why. Check the actual selection in history.
4. Attach inputs to the Task or its parent. Parent inputs flow to children; dependency outputs flow to downstream Tasks. Siblings do not share outputs merely by being siblings, and files do not automatically cross Shared sessions.
5. Start the Discovery Engine. Inspect `New` → `Executing` → `Validating` → `Complete` and validation comments. Failed validation can lead to `Incomplete`, retries, or `Needs User Attention`.

6. Confirm that T3 actually discovers and reads inherited files through `GetResourceContext` / `PreviewResource` . File contents are not automatically injected into its prompt. Root-level output aggregation and downstream input inheritance are also different mechanisms.
7. Validate file **content**, not just `Complete` or an Asset ID. Required Tasks that fail, remain on hold, or need human attention do not pass the lab. The Engine ending a session does not prove that all research succeeded.

Pass criteria: Relationships, actual execution, validation comments, and input/output Asset links are verified for R0/T1/T2/T3. Required outputs match S02, and the T3 and R0 reports are readable.

Autonomy extension: After the baseline passes, optionally give one bounded objective to a separate session and observe cognition create subtasks. Permit only the eight synthetic candidates; prohibit additional cloud deployments and physical experiments. This contrasts structured execution, guided exploration, and autonomous research without promising identical outcomes, duration, or cost. ^[S29]

L07 — Change constraints and provide human feedback

Goal: Preserve evidence and history while testing a new hypothesis and re-evaluating the result.

Steps:

1. **Stop the Engine after all baseline Tasks finish.** Inspect remaining executions and let them finish or normally cancel the intended operation when necessary. Preserve the baseline once no work is still running.
2. Test "lowering the cost ceiling to 3.00 leaves only DELTA." With the **Engine off**, configure all follow-up Tasks, dependencies, and validation requirements below in the same Shared session. Do not overwrite the baseline. This is constraint testing on synthetic data, not a real materials experiment.

Task	Depends on	Output and required validation
T4	T3	Actual <code>rank_cost_revision</code> run, new operation ID, <code>ranking-cost3.json</code> , only DELTA eligible at 68.6
T5	T3, T4	<code>comparison.md</code> : explain three → one eligible, retain denominator 5, preserve ZETA's unresolved gap

3. Put the following in the Task Description and comments.

Lower only the hard cost ceiling from 5.00 to 3.00 USD/kg.
 Keep all other constraints and the cost-normalization denominator at 5.
 Actually run thermal-ranking's `rank_cost_revision` and save a separate result.
 Preserve prior evidence and outputs. Do not invent corrosion results or measurements.
 Distinguish the hypothesis result from evidence that still needs to be acquired.

4. Check the completed configuration and absence of duplicate running work, then **Start the Engine**. Confirm that earlier files/history persist and the new Tasks execute. Restart rebuilds working memory. **Stop does not cancel already-running agents/tools.** Do not arbitrarily change an executing Task's state or submit the same work again. ^[S13]

- After T4/T5 finish, Stop the Engine and inspect remaining work. Download the original `ranking-cost3.json` and run:

```
python3 scripts/verify_ranking.py \
  --input '<downloaded-ranking-cost3.json>' --case cost3
```

- If file checks or Task validation fail, fix the cause. Use **Stop → inspect remaining executions → edit → Start** for corrections/retries as well. Do not remove correct GAMMA/ZETA requirements merely to obtain a pass.

Pass criteria: A separate operation ID, `cpu1ab`, original file, and passing `cost3` check confirm only DELTA at 68.6. ZETA is `excluded` on cost but retains its missing-evidence record. Preserve baseline results/history and do not present counterfactual assumptions as observations.

L08 — Collaboration and project access control

Goal: Verify collaboration within a project and boundaries around other projects and linked resources. [\[S19\]](#) [\[S20\]](#)

Steps:

- An administrator prepares actual test users/groups and an unassigned comparison project for denial checks. **Without another user or comparison project, defer the corresponding verification.** Two browser windows using the same identity do not verify multi-user authorization. Obtain approval for the scope/cost before creating an additional comparison project.
- Assign roles at the exact ARM scope of existing project `thermalhol`. Also inspect inherited Workspace/RG/subscription permissions.

Persona	Project role	Linked-resource roles as needed
Authoring/running researcher	Microsoft Discovery Project Contributor (Preview)	Tools Reader and Chat Model Reader for authoring; required Storage Container/Blob and Bookshelf roles
Results reviewer	Microsoft Discovery Project Reader (Preview)	Storage Container Reader + Storage Blob Data Reader; optionally Bookshelf Index Data Reader

- Scope each assignment to approved resources. **Project Contributor does not create new Projects or assign roles.** Storage Container metadata access does not replace Blob-content access.
- Verify that the researcher can open the same project's Shared session, Tasks, and outputs and make permitted changes. Verify that the reviewer can read but cannot create, change, or run work.
- Verify denial of access to an unassigned project. If the test user inherits broad access, do not claim isolation passed; correct the test scope.
- Distinguish running an existing agent from authoring one. Tools Reader/Chat Model Reader are not always required solely to run an administrator-created agent. Running the Engine with its platform `gpt-5-4` does not necessarily need a separate Chat Model Reader assignment.

7. Linked tool/model access is checked when an agent is created or updated. Do not assume revoking the user's linked-resource role immediately prevents an existing agent from using that resource; review/update affected agents.

Pass criteria: Another user's allowed/denied outcomes and assignment scopes are recorded. Some Workbench status operations require Workspace access even for project-only users; do not resolve every error by granting broad Platform roles.

L09 — Preserve results, review cost and shut down

Goal: Leave an auditable result and distinguish running work from always-on costs when ending the lab.

Steps:

1. Retain `research-report.md`, `baseline-summary.md`, `comparison.md`, both actual computation JSON files, and content-check results. Keep them separate from local reference JSON, and distinguish content validation from Azure execution evidence.
2. Complete the reproducibility record.

Category	Record
Scope	Workspace/Project/Shared session/Task IDs
Inputs	Document IDs/versions, CSV, KB name/version/index status
Configuration	Model deployment, actual agent version, tool-definition version/image digest, node pool
Actual execution	Operation ID, time, terminal state, logs
Outputs	Storage Asset IDs, downloaded files, decisions/scores, validation comments
Limitations	Synthetic data, unperformed experiments, missing evidence, variable model responses

3. Stop the Engine and separately inspect running agents/tools. Cancel only the intended operations if needed. **Cancellation itself can be a billable runtime action.**
4. In Cost Management, inspect the lab RG, each MRG, **Microsoft Discovery - User Messages**, models, compute, and storage. Do not estimate the bill from user-prompt count alone. [\[S24\]](#)
5. When retention ends, agree deletion scope with the administrator. Exclude shared resources and first check running work and files to preserve.
6. Distinguish **reference deletion** in Storage Asset → Storage Container order from actual Blob deletion. Removing references does not delete the backing Blob data/account. Clean up Bookshelf, Workspace, Supercomputer, and MRGs according to documented lifecycle and actual dependencies. [\[S16\]](#) [\[S17\]](#)
7. Record Engine state, remaining operations, node state, always-on resources, and retention/deletion outcomes separately. Closing the browser or setting min=0 does not make all costs zero.
8. To reuse reviewed knowledge in another session, explicitly attach the Assets or follow a supported KB creation/update procedure. Do not assume a successful Task automatically reindexes the existing Bookshelf.

Pass criteria: The evidence→execution→output chain is retained and remaining work, resources, and cost exposure are clear. Do not claim collaboration verification if L08 was not performed.

E01 — Code-environment and Hybrid Tools

Goal: Compare a fixed Action with agent-generated code in the same compute environment. This is an optional developer extension after the core route. [\[S21\]](#) [\[S22\]](#)

Steps:

1. Copy the existing Action-only definition instead of overwriting it. Name it `thermal-hybrid`, version `1.1.0`, and retain the validated image and CPU limits.
2. Add this top-level `code_environments` array to the JSON definition. If using YAML, convert it to the corresponding JSON when registering.

```
{
  "code_environments": [
    {
      "language": "python",
      "command": "python \"{{scriptName}}\"",
      "description": "Python standard library only. Analyze original synthetic training data; no network access or new package installation is requested.",
      "infra_node": "worker"
    }
  ]
}
```

3. This is an **addition, not a complete tool definition**. Retain `infra` and `actions`. Combining fixed Actions with a Code-environment makes a Hybrid Tool.
4. Attach it to a separate test agent. Ask it to write and execute code on `/app/materials.csv` that calculates GAMMA's margin against the conductivity requirement and reads ZETA's `corrosion_status`. Do not request network access or package installation. Instructions are not a network control; enforce actual restrictions through operational policy.
5. Verify generated code, the actual operation ID/logs, `175 - 180 = -5`, and `corrosion_status = "unknown"`. **The CSV column exists; its value is unknown.** `corrosion_result` in the computation JSON's `missing_evidence` is a derived missing-evidence label, not a CSV column name.

Pass criteria: A new Hybrid definition and actual generated-code execution evidence exist. Do not conflate Code-environment, Foundry Code Interpreter, and remote MCP execution runtimes.

E02 — External tools and scientific-model integration

Goal: Verify an extension through a public-documentation MCP example, then choose the appropriate integration pattern for scientific models. Separately verify connected-service feature status, model support,

and organizational policy. [\[S10\]](#) [\[S22\]](#) [\[S26\]](#)

Steps:

1. Create a separate **DocsConnector** agent in the same Discovery project, isolated from research-data agents.
2. Open advanced settings for **the Foundry project/agent backing that Discovery agent**. Do not create an unrelated Foundry project.
3. In Foundry Tools → Add a new tool → Custom → MCP, configure name **MicrosoftLearnPublic**, endpoint **https://learn.microsoft.com/api/mcp**, and Authentication **Unauthenticated**. This setting is specific to the public Learn server, not a template for authenticating private MCP services.
4. Ask only the public question “How do Storage Containers and Storage Assets differ in Microsoft Discovery?” Do not send lab documents, tenant/subscription details, or research data. Follow normal user-approval prompts for tool use.
5. Verify **actual MCP call logs and official source URLs**. Confirm the same agent's connection is reflected in Discovery. If the UI, permissions, or connection feature is unavailable, defer this extension only.

Asset to integrate	Appropriate pattern
Existing computational binary/simulator	Container Action → Supercomputer
Agent-authored analysis code	Code-environment / Hybrid → Supercomputer
Already-hosted REST model	OpenAPI schema + approved authentication/connection
Tools exposed by an MCP server	Foundry MCP connection + approval/policy
LLM used for agent reasoning	Select a Workspace Chat Model Deployment; distinguish Engine validation

Property prediction by scientific/large quantitative models (LQMs) is different from LLM language reasoning. Integrate scientific models through the Container/OpenAPI/MCP patterns above as appropriate. This lab's Python scorer is not a physical simulator or a trained scientific model. If adding public-literature search, use approved search/scientific-database connections and verify sources, publication dates, and usage rights.

Pass criteria: Actual calls and sources demonstrate MCP use, not just a verbal claim. Reading the pattern table does not mean every OpenAPI integration or partner model was deployed and evaluated.

A01 — Cost, quota and conflicting documentation

There are two cost layers: Discovery **runtime User Messages** and separate Azure resource/model consumption. GET/list/status/log operations are not billed as Discovery runtime actions, but always-on infrastructure can keep accruing cost while you read. Azure Budgets generally alert; they are not hard spending caps. [\[S24\]](#)

Preparation item	Planning baseline
GPT-5.4	Check at least 500,000 TPM total , following the documentation's Important note; verify allocations per deployment
Bookshelf models	Use GPT-5.2 200,000 TPM , GPT-5 Mini 2,000,000 TPM , and text-embedding-3-small 2,000,000 TPM as preparation baselines, then confirm actual creation/search requirements
Always-on Bookshelf infrastructure	The indexing how-to specifies Search S1 with 2 replicas, SQL Hyperscale Gen5 with 20 vCores, and ACA Small E4. Four documents still incur fixed cost
Indexing memory	Resolve the difference between Small E20s_v6 (160 GB) in the how-to and D48s_v6 (192 GB) in the quota page before deployment
Computation pool	D4s_v6-family CPU pool; no GPU needed. Separate from the indexing pool
Other costs	Workspace/Project Cosmos DB, Storage, Registry/ACR Tasks, networking, and managed infrastructure

Check model quota for the subscription, region, model, and deployment type together. The documented default is Global Standard. If residency requirements lead to Data Zone Standard or another type, recheck support and the corresponding quota. [\[S05\]](#)

The following are **observed documentation differences** and this guide's treatment. Do not copy contradictory statements into a single supposedly deterministic setup.

Topic	Documentation difference	Guide decision
GPT-5.4 quota	Summary 450,000 vs Important note 500,000 [S05]	Use 500,000 as a conservative preparation baseline; verify actual deployment quotas
Indexing SKU	E20s_v6 vs D48s_v6 [S05] [S06]	Confirm supported SKU/memory with the administrator/product team; do not substitute the small computation pool
Reindexing	Concept page says delete/reindex; how-to describes incremental enrichment plus a full graph rebuild [S06] [S07]	Require the first full index only. Confirm the deployed version before update/deletion decisions
Model response controls	Temperature recommendations in model selection vs unsupported reasoning-model controls [S09] [S10] [S25]	Leave temperature/top_p unset for this reasoning-model route
Automatic model creation	Default automatic-model wording vs manual-deployment quickstart [S03] [S13]	Check actual <code>gpt-5-4</code> ; create only if missing
Billing unit	User Message/operation wording conflicts with the 10-operation conversion example [S24]	Do not multiply raw API-call count directly by a unit price. Confirm the regional meter, contract and actual bill
Storage hierarchy	Workspace-child wording in a concept table vs actual RG-level resource IDs [S16] [S17]	Use RG-level Storage Container/Assets referenced by the project

Topic	Documentation difference	Guide decision
Regional scope	Infrastructure quickstart's production-region list differs in scope from the tool-deployment page including East US 2 [S03] [S33]	Use Sweden Central, present in both. Do not infer full-stack availability from a single tool resource

Before deployment, review actual region, SKUs, quantities, retention time, and subscription terms in the [Azure Pricing Calculator](#). Product/model GA does not make every model, SDK, API, or connection feature GA. Preserve `preview` status when it appears in an example API version.

A02 — Troubleshooting and completion

Symptom	Check
Provider is Registered but resource types are unavailable	Service enablement, supported types/API; separate from spending/RBAC approval
Engine cannot start	<code>gpt-5-4</code> , actual model quota, specialist agents and tools
Task stays New	Dependencies, Engine state, execution capacity. Do not force Complete
Repeated Incomplete / Needs User Attention	Validation comments, missing tools/inputs, impossible requirements; fix the cause
Output link returns 403	User Blob data role, firewall/VPN and CORS; independent of managed-identity success
Model/tool unavailable during agent creation	Reader access on that linked resource; project access alone can be insufficient
Indexing fails	Format, source path, managed identity, memory pool, model quota
Tool creation or image pull fails	Definition content file/version, actual image reference, Registry connectivity and managed-identity pull role
Ranking JSON check fails	Original tool file vs summary, baseline/cost3 selection, data/tool version, candidate/decision/score mismatch. Do not overwrite it with the local answer
Agent lacks earlier chat context	Chat/Task separation; specify Task inputs and Description
Restart triggers another warmup	Working memory rebuild; verify retained results/history
Costs continue after computation	Running jobs, system pool, always-on Bookshelf/Workspace infrastructure

Core completion: Evidence exists for L00–L07 and L09, with matching values and citations. L08 passes only with another user's authorization checks. Record E01/E02 separately. **Label an unexecuted capability “explained/prepared only.”**

Local checks of the supplied package do not change Azure.

```
npm ci
npm run build:guide
npm test
```

Use `npm run check:guides` for browser rendering checks. The supplied script uses **an installed Google Chrome in headless mode**. If Chrome is missing, follow an approved installation process; installing Chromium alone does not satisfy `channel: chrome`.

A03 — Official sources

Checked on 2026-09-25. Both editions use identical Source IDs/URLs, Lab IDs, data, and expected results. Product UI labels remain in English for discoverability.

ID	Official documentation
S01	GA announcement
S02	Service architecture
S03	Infrastructure quickstart
S04	Provider registration
S05	Quota reservations
S06	Create Bookshelf and index a Knowledgebase
S07	Bookshelf concepts
S08	Agents and shared sessions quickstart
S09	Create agents
S10	Agent types
S11	Discovery Engine
S12	Tasks and shared sessions
S13	Cognition
S14	Build shared sessions with cognition
S15	Files and storage assets
S16	Storage containers and assets
S17	Manage storage containers

ID	Official documentation
S18	Configure network security
S19	Project RBAC concepts
S20	Configure project access
S21	Create a tool definition
S22	Tools and model integration
S23	Publish a tool image to ACR
S24	Discovery billing
S25	Select models for agents
S26	Microsoft Learn MCP in Foundry
S27	Debug task execution
S28	Discovery cloud and Discovery app
S29	Advanced shared-session patterns
S30	Discovery Studio
S31	Discovery agents and versioning
S32	Supercomputer job submission, status and cancellation
S33	Deploy a tool to Discovery: Portal fields and REST envelope